

Luca Sepe

Platform Architect

Control-plane software, API machinery, Go, and infrastructure automation

Italy - remote-first

- luca.sepe@gmail.com
- linkedin.com/in/lucasepe
- github.com/lucasepe
- leanpub.com/u/lucasepe

I work with CTOs and platform teams building Kubernetes products and internal platforms. I contribute architecture, hands-on implementation, and structured handover when APIs, controllers, automation, and ownership boundaries are becoming difficult to evolve.

When I am useful

- A Kubernetes platform is growing faster than its architecture.
- CRDs and APIs are becoming difficult to evolve safely.
- Controller behavior is hard to reason about under failure.
- A critical platform component needs careful design and implementation.
- A CTO needs implementation-oriented engineering judgment.

Representative Engineering Work

- Designed and implemented core components of a Kubernetes-native platform.
- Built JSON Schema to CRD conversion tooling.
- Implemented reusable dynamic controller behavior with dynamic clients and unstructured resources.
- Developed custom Kubernetes API servers and infrastructure software in Go.
- Worked across API design, reconciliation behavior, platform tooling, and operational handover.

Engagement model

- Focused architecture discovery before a larger investment.
- Hands-on implementation sprint for critical platform components.
- Kubernetes API, controller, and control-plane review.
- Documentation, mentoring, and structured handover.
- Focused engagements with a clear outcome, explicit handover, and no expectation of long-term consultant dependency.

Business outcomes

- Clearer API, platform, and ownership boundaries.
- Reduced architectural risk before implementation becomes expensive.
- Production-oriented components instead of fragile prototypes.
- Internal ownership through documentation and structured handover.
- More explicit operational behavior and less external dependency.

Technical Author

The Kubernetes Platform Engineer Interview Guide

Advanced Controllers, Syncers, Multi-Tenancy, API Scalability, and Staff-Level System Design.

Go Apps on Kubernetes

Production practices for Go services on Kubernetes.

Kubernetes Context Engineering

Operational context for SRE workflows and AI tools.

[Complete catalogue on Leanpub](#)

Working principles

- Clarity over cleverness.
- Design APIs before controllers.
- Keep architecture grounded in implementation.
- Treat documentation and handover as part of delivery.
- Leave the internal team able to own the result.

Building or evolving a Kubernetes platform? Start with the difficult engineering problem. We can then decide whether a focused review, discovery sprint, or implementation is the useful next step.